

Entregables T2 — Evolución temporal (Sebastián)

items 3 y 4: integración directa (RK4) y acción del exponencial (Krylov)

Sebastián

3 de junio de 2026

Documento de revisión autocontenido. Los mismos fragmentos se \input en el documento maestro mpemba_cuantico.tex.

Índice

1 Evolución temporal (a): integración directa con RK4	1
1.1 Explicación del framework	1
1.2 Discretización y método numérico	1
1.3 Implementación serial	1
1.4 Justificación de la paralelización: OpenMP	2
1.5 Comparación serial vs paralelo	2
1.6 Resultados y validación	2
2 Evolución temporal (b): acción del exponencial por Krylov	4
2.1 Explicación del framework	4
2.2 Discretización y método numérico	4
2.3 Implementación serial	4
2.4 Justificación de la paralelización: OpenMP	4
2.5 Comparación serial vs paralelo	5
2.6 Comparación de método (a vs b): el resultado central	6
2.7 Resultados y conclusión del método (b)	6

1 Evolución temporal (a): integración directa con RK4

1.1 Explicación del framework

La tarea de *evolución temporal* consiste en obtener la trayectoria del operador densidad $\rho(t)$ que resuelve la ecuación maestra de Lindblad $\dot{\rho} = \mathcal{L}[\rho]$ (ec. (??)) a partir de una preparación ρ_0 , para trazar las curvas de distancia al equilibrio $\mathcal{D}(\rho_t || \rho_{ss})$ que revelan (o descartan) el efecto Mpemba. El método (a) es la vía **directa**: integrar la ecuación diferencial paso a paso, sin construir el superoperador $\mathbb{L}$ de dimensión $d^2 \times d^2$, aplicando en su lugar la acción *matrix-free* $\mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} (L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\})$.

1.2 Discretización y método numérico

Se emplea el método de **Runge–Kutta clásico de 4.º orden** (RK4), un integrador explícito de un paso. Para $\dot{\rho} = \mathcal{L}[\rho]$ y paso Δt :

$$\begin{aligned} k_1 &= \mathcal{L}[\rho_n], & k_2 &= \mathcal{L}[\rho_n + \frac{\Delta t}{2} k_1], & k_3 &= \mathcal{L}[\rho_n + \frac{\Delta t}{2} k_2], \\ k_4 &= \mathcal{L}[\rho_n + \Delta t k_3], & \rho_{n+1} &= \rho_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

El error local es $\mathcal{O}(\Delta t^5)$ y el global $\mathcal{O}(\Delta t^4)$. Cada paso requiere *cuatro* evaluaciones del Liouvilliano; cada evaluación, para la cadena de Ising disipativa, son $2 + 3N_J$ productos de matrices densas $d \times d$ (con $N_J = 2N$ operadores de salto), de coste $\mathcal{O}(d^3)$ cada uno. El coste por paso es por tanto $\mathcal{O}(N d^3)$.

1.3 Implementación serial

El núcleo está en `common/lindblad.hpp` (acción `apply_lindblad`, modelo `build_ising`, estado de Gibbs por tiempo imaginario) y el integrador en `a_integracion_directa/src/rk4_evolution.cpp`. La preparación inicial es un estado de Gibbs a $T_0 = 5$ y la referencia de relajación el estado de Gibbs del baño a $T = 0,8$. El *hotspot* absoluto es el producto de matrices densas `matmul`.

1.4 Justificación de la paralelización: OpenMP

El patrón de cómputo de la integración directa es: *miles de pasos secuenciales* (dependencia temporal estricta: ρ_{n+1} necesita ρ_n), cada uno dominado por productos de matrices densas que reutilizan los mismos datos. La dependencia temporal **impide** paralelizar entre pasos; el paralelismo disponible está *dentro* de cada paso, en el `matmul`. Ese es un patrón de **memoria compartida** con datos regulares y de tamaño moderado ($d \times d$ cabe en caché/RAM de un nodo): el paradigma natural es **OpenMP**, paralelizando el bucle de filas del `matmul` (`#pragma omp parallel for`). MPI sería contraproducente aquí (distribuir matrices pequeñas introduce comunicación de halo en cada uno de los miles de pasos); CUDA daría más ancho de banda pero no hay GPU disponible en la máquina de prueba (se discute como trabajo futuro en §2 y en la tabla ??). La corrección de la versión paralela se verifica de forma exacta: serial y multinúcleo producen el *mismo* estado final (coincidencia a precisión de máquina del observable D_{HS} final).

1.5 Comparación serial vs paralelo

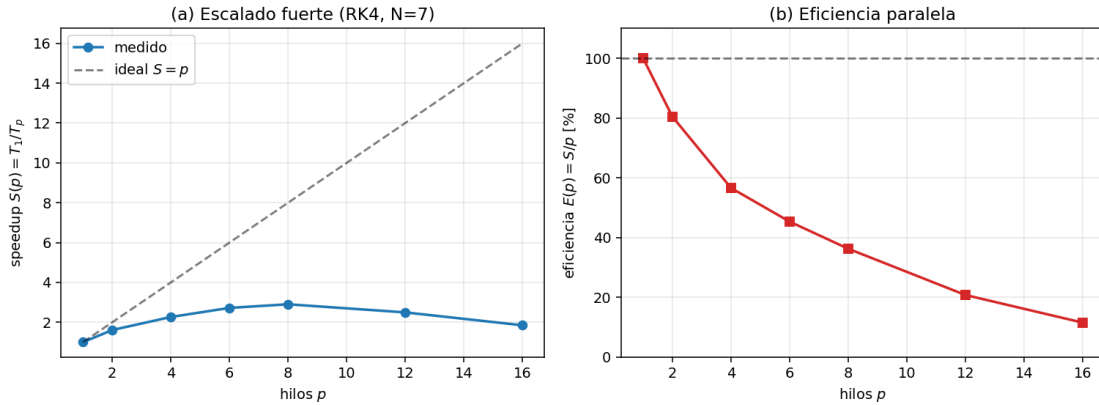


Figura 1: Escalado fuerte del integrador RK4 (cadena de Ising disipativa $N = 7$, $d = 128$) en un AMD Ryzen 7 7730U (8 núcleos físicos / 16 hilos). (a) speedup $S(p) = T_1/T_p$ frente al ideal $S = p$; (b) eficiencia paralela $E(p) = S/p$.

El speedup es **sublineal** y satura cerca del número de núcleos físicos: se alcanza $S_{\max} \approx 2,9$ con 8 hilos, con eficiencia del 36 % a 8 hilos. El ajuste a la **ley de Amdahl** $S(p) = 1/[f + (1-f)/p]$ da una fracción serie efectiva $f \approx 0,25$. Las causas de la sublinealidad son típicas de este patrón: (i) el integrador hace *fork/join* de OpenMP en cada uno de los muchos `matmul` pequeños, con sobrecoste acumulado; (ii) el problema es *memory-bandwidth bound* (las matrices se releen de memoria), de modo que los hilos compiten por el ancho de banda compartido; (iii) las asignaciones de memoria temporales y las operaciones vectoriales fuera del `matmul` son seriales. El salto de 8 a 16 hilos

aporta poco porque los 16 hilos son *hyperthreads* sobre 8 núcleos físicos, sin ancho de banda adicional.

Para ver *directamente* la mejora —y no solo el speedup adimensional— la Figura 2 enfrenta el **tiempo de cómputo en serie (1 hilo) y en paralelo (8 hilos)** en función del tamaño del sistema N (la malla del operador, $d = 2^N$), a igual número de pasos. Ambas curvas crecen como $\sim d^3$, pero la paralela queda sistemáticamente por debajo: la *separación* entre las dos es la ganancia, y crece con la malla porque a mayor d el **matmul** domina y amortiza mejor el coste de *fork/join*.

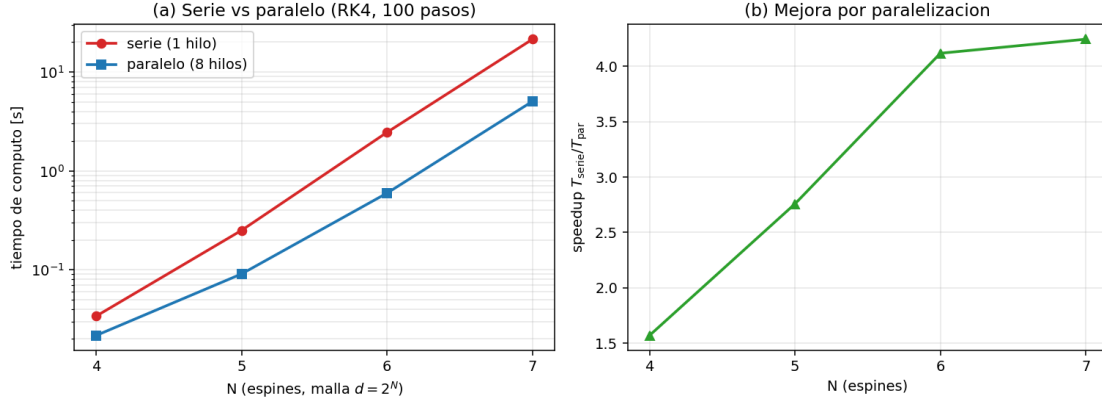


Figura 2: Tiempo de cómputo **serie vs paralelo** del RK4 frente al tamaño del sistema N (malla $d = 2^N$), a 100 pasos fijos. (a) las dos curvas (1 hilo en rojo, 8 hilos en azul, escala log) muestran el mismo coste $\sim d^3$ desplazado: el hueco entre ellas es la mejora. (b) el speedup $T_{\text{serie}}/T_{\text{par}}$ resultante crece con la malla (de $\sim 1,6$ en $N = 4$ a ~ 4 en $N = 7$): cuanto mayor es d , más domina el **matmul** y mejor se amortiza el *fork/join*, aunque siempre sublineal frente al ideal de 8.

1.6 Resultados y validación

Correctitud y orden del método. La figura 3 valida el integrador frente a la evolución espectral *exacta* $e^{t\mathcal{L}}\rho_0$ (suma de modos): el error a tiempo fijo decae como Δt^4 , confirmando el **orden 4** del RK4 (pendiente medida $\approx 4,04$).

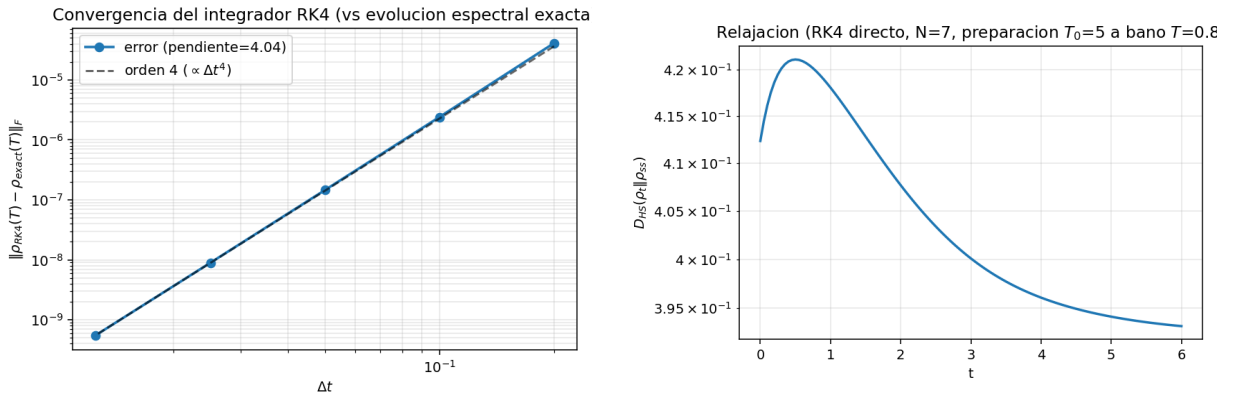


Figura 3: Izquierda: convergencia de orden 4 del RK4 frente a la solución exacta. Derecha: curva de relajación $D_{HS}(t)$ producida por el método (preparación caliente $T_0 = 5$ relajando al baño $T = 0,8$).

Coste con el tamaño. La figura 4 confirma el escalado $\mathcal{O}(d^3)$ por paso esperado del producto de matrices densas: al pasar de N a $N + 1$ ($d \rightarrow 2d$) el coste por paso se multiplica por ~ 8 . Es justamente este crecimiento el que vuelve inviable la representación densa de ρ para N grande y motiva las trayectorias (§??) en el régimen de muchos cuerpos.

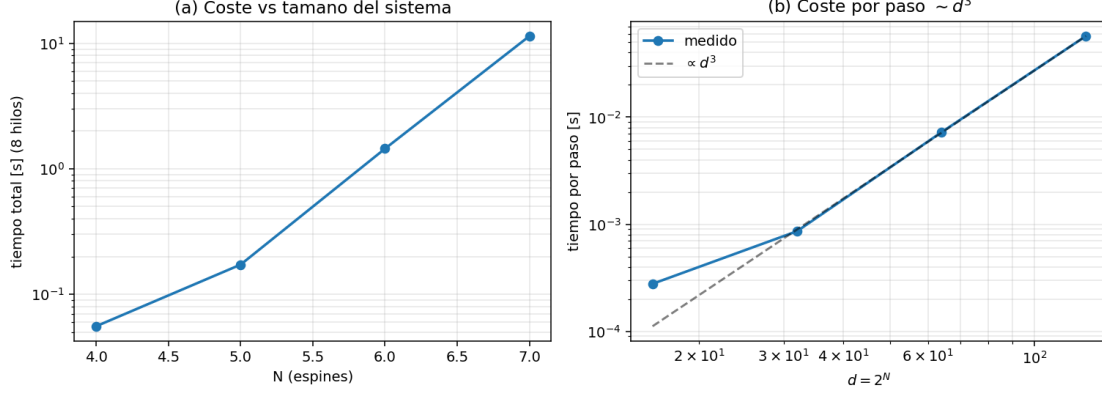


Figura 4: Coste del método con el tamaño del sistema (8 hilos). (a) tiempo total; (b) tiempo por paso frente a la referencia $\propto d^3$.

Conclusión del método (a). La integración directa por RK4 es *simple, exacta a orden 4 y robusta*, y se paraleliza de forma natural con OpenMP a nivel del producto de matrices. Su limitación es doble: el coste $\mathcal{O}(d^3)$ por paso y la necesidad de pasos pequeños por estabilidad/precisión, que obligan a muchas evaluaciones del Liouvilliano. El método (b) (acción del exponencial, §2) ataca precisamente este segundo punto.

2 Evolución temporal (b): acción del exponencial por Krylov

2.1 Explicación del framework

El método (b) resuelve la misma evolución $\dot{\rho} = \mathcal{L}[\rho]$ pero por una vía distinta: en lugar de integrar paso a paso, aproxima directamente la **acción del exponencial del Liouvilliano**,

$$\rho(t + \tau) = e^{\tau \mathcal{L}}[\rho(t)],$$

que es la solución *exacta* en un paso de tamaño τ . La dificultad es que $e^{\tau \mathcal{L}}$ es un objeto de dimensión $d^2 \times d^2$ imposible de formar. La idea de Krylov es proyectar la acción sobre el subespacio $\mathcal{K}_m = \text{span}\{\rho, \mathcal{L}\rho, \dots, \mathcal{L}^{m-1}\rho\}$, de dimensión $m \ll d^2$, donde el exponencial es trivial de calcular.

2.2 Discretización y método numérico

Se construye con **Arnoldi** una base ortonormal $\{Q_0, \dots, Q_{m-1}\}$ de $\mathcal{K}_m$ (cada Q_i es un operador $d \times d$; el producto interno es el de Frobenius $\langle A, B \rangle = \text{Tr}(A^\dagger B)$) y la matriz de Hessenberg $H_m \in \mathbb{C}^{m \times m}$ que representa $\mathcal{L}$ en esa base. Entonces

$$e^{\tau \mathcal{L}}[\rho] \approx \|\rho\|_F \sum_{i=0}^{m-1} [e^{\tau H_m}]_{i,0} Q_i,$$

donde $e^{\tau H_m}$ se calcula sobre la matriz *pequeña* $m \times m$ por *scaling-and-squaring* con serie de Taylor. El coste por bloque es de m aplicaciones de $\mathcal{L}$ (Arnoldi) más operaciones $\mathcal{O}(m d^2)$ y un $e^{\tau H_m}$ de coste $\mathcal{O}(m^3)$ despreciable. La ventaja decisiva: para τ dentro del radio de convergencia, $m \sim 20\text{--}30$ basta para precisión de máquina y *un solo bloque avanza un paso de tiempo grande*, donde RK4 necesitaría decenas o cientos de pasos pequeños.

2.3 Implementación serial

En `b_accion_exponencial/src/krylov_evolution.cpp`: Arnoldi sobre la acción `apply_lindblad` (`common/lindblad.hpp`), exponencial de matriz pequeña `small_exp` (Taylor de 18 términos + *squaring*), y avance por bloques de tamaño τ . Se contabiliza el número de aplicaciones de $\mathcal{L}$ como medida de *trabajo* independiente de la máquina.

2.4 Justificación de la paralelización: OpenMP

Como en el método (a), el *hotspot* es la acción $\mathcal{L}[\cdot]$ (el producto de matrices densas), llamada m veces por bloque. La ortogonalización de Arnoldi añade productos internos de Frobenius de coste $\mathcal{O}(d^2)$, mucho menores que el $\mathcal{O}(d^3)$ del `matmul`. Por tanto el patrón sigue siendo de **memoria compartida** dominado por `matmul`, y el paradigma natural vuelve a ser **OpenMP** sobre el mismo bucle. (En el régimen de muchos cuerpos, donde d^2 no cabe en memoria, la acción de Krylov se paraleliza con MPI distribuyendo el vector $|\rho\rangle$; aquí, con ρ denso en un nodo, OpenMP es lo adecuado.) La corrección se verifica de dos formas: serial y paralelo dan el mismo resultado, y el resultado coincide con el del método (a) (RK4) a todas las cifras: $D_{HS}^{\text{final}} = 0,4326030367$ para $N = 6$, idéntico por ambos métodos (referencia exacta por evolución espectral: $D_{HS}^{\text{exacto}} = 0,42411283$).

2.5 Comparación serial vs paralelo

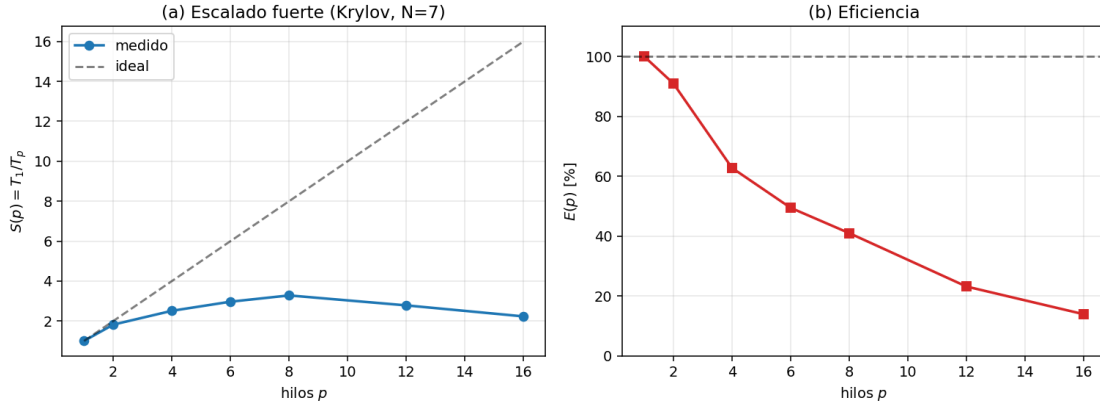


Figura 5: Escalado fuerte del método de Krylov (Ising disipativo $N = 7$, $d = 128$). El comportamiento es análogo al del método (a): speedup sublineal que satura en los núcleos físicos, por las mismas causas (fork/join del `matmul`, *memory-bandwidth bound*, hyperthreading sin ancho de banda extra). Pico $S_{\text{max}} \approx 3,3$, eficiencia 41 % a 8 hilos (ligeramente mejor que RK4: hay menos llamadas a `matmul` por unidad de trabajo).

La Figura 6 muestra la mejora de forma directa: el **tiempo de cómputo en serie (1 hilo) frente al paralelo (8 hilos)** en función del tamaño del sistema N (malla $d = 2^N$), a igual paso τ y misma dimensión de Krylov m . Igual que en (a), las dos curvas crecen como $\sim d^3$ y el hueco entre ellas —la ganancia— se ensancha con la malla.

2.6 Comparación de método (a vs b): el resultado central

La ventaja de Krylov no está en el escalado paralelo (idéntico al de RK4) sino en el **trabajo necesario para una precisión dada**. La figura 7 mide el error del observable D_{HS} final frente a una referencia de alta precisión, en función del número de aplicaciones de $\mathcal{L}$ (la operación cara, independiente de la máquina):

La interpretación física/numérica: RK4 converge *algebraicamente* (el error baja como Δt^4), mientras que Krylov converge *super-algebraicamente* (casi exponencial en m) porque captura la

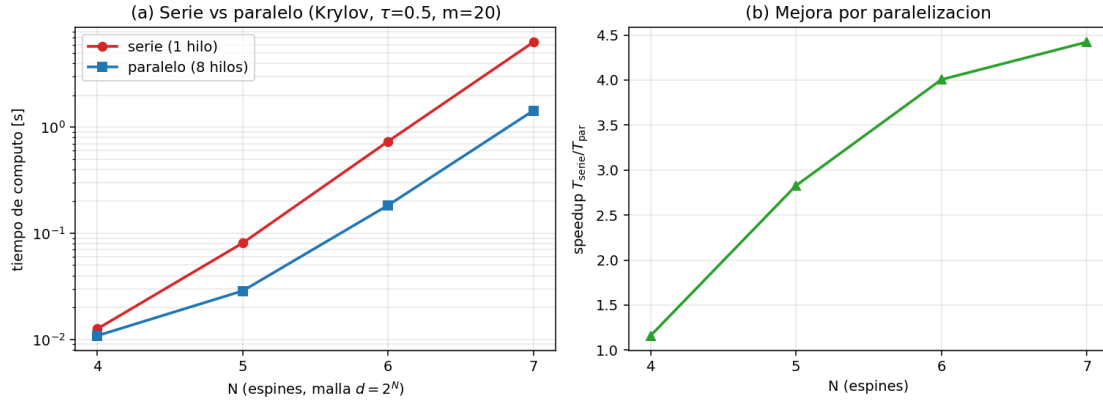


Figura 6: Tiempo de cómputo **serie vs paralelo** del método de Krylov frente al tamaño del sistema N (malla $d = 2^N$), a $\tau = 0,5$ y $m = 20$ fijos. (a) curvas de 1 hilo (rojo) y 8 hilos (azul) en escala log; (b) speedup $T_{\text{serie}}/T_{\text{par}}$ resultante. El patrón es el mismo que el de RK4 porque el hotspot paralelo —el `matmul` de la acción de $\mathcal{L}$ — es común a ambos métodos.

acción exacta del exponencial dentro del subespacio. En este sistema disipativo *no rígido*, ambos métodos son de trabajo comparable para precisiones moderadas, pero Krylov presenta tres ventajas estructurales: (i) a igual trabajo es $\sim 25\times$ más preciso; (ii) usa pasos de tiempo $5\times$ mayores ($\tau = 1,0$ frente a $\Delta t = 0,2$) manteniéndose estable y exacto; (iii) es **incondicionalmente estable**, mientras que RK4 tiene un límite de estabilidad $\Delta t < 2,8/|\lambda_{\text{max}}|$. Esta ventaja se vuelve *decisiva* para generadores *rígidos* (escalas de energía grandes o disipación fuerte) y para alcanzar el estado estacionario, donde el límite de estabilidad obliga a RK4 a dar muchísimos pasos diminutos. RK4 sigue siendo preferible cuando se necesita la trayectoria *densamente* muestreada en el tiempo o cuando el Liouvilliano depende del tiempo.

2.7 Resultados y conclusión del método (b)

La figura 8 muestra la curva de relajación obtenida con pasos *grandes* ($\tau = 0,25$), inalcanzables para RK4 a esa precisión. Resumen de la comparación a–b:

- Misma física (resultados idénticos), misma paralelización (OpenMP sobre `matmul`, escalado sublineal equivalente).
- Krylov (b): mucho menos trabajo para alta precisión y pasos de tiempo grandes; ideal para llegar al estacionario o muestrear pocos tiempos.
- RK4 (a): más simple, robusto, y mejor para trayectorias densas o generadores dependientes del tiempo.

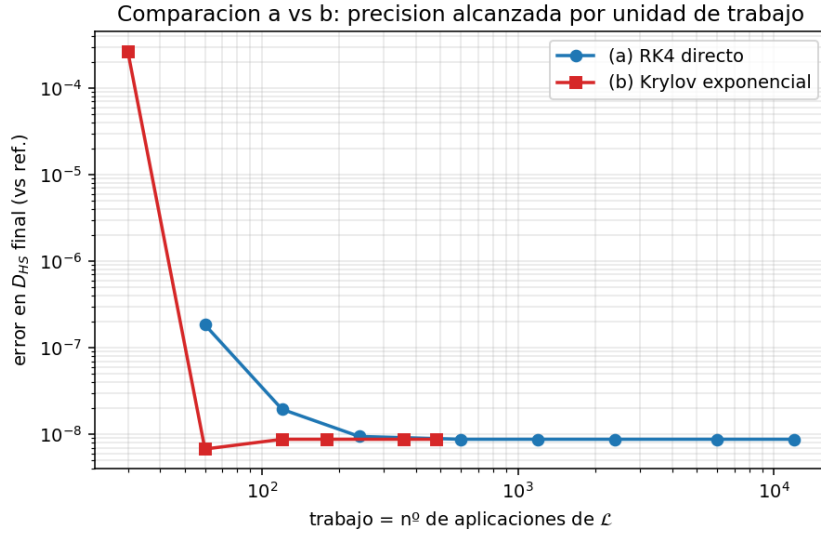


Figura 7: Error del observable D_{HS} final frente a la referencia exacta (evolución espectral), en función del trabajo. **A igual trabajo (≈ 60 aplicaciones de $\mathcal{L}$), Krylov es $\sim 25\times$ más preciso** (7×10^{-9} frente a 2×10^{-7} de RK4). El punto Krylov de $m = 10$ (no convergido, 2×10^{-4}) cae cinco órdenes de magnitud al pasar a $m = 20$: convergencia *super-algebraica* en m , frente a la algebraica ($\propto \Delta t^4$) de RK4. El suelo $\sim 9 \times 10^{-9}$ es la precisión de la propia referencia.

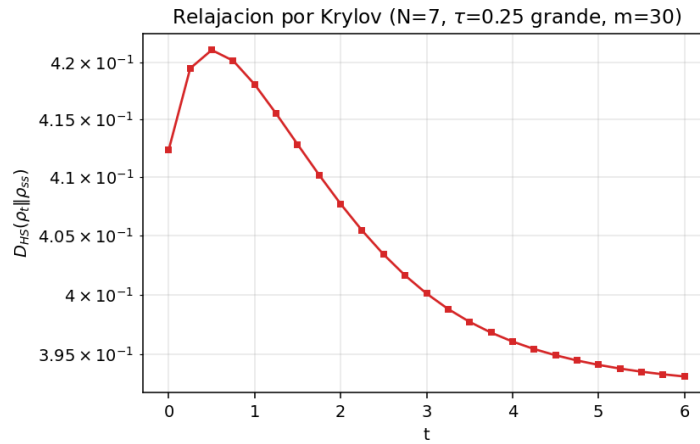


Figura 8: Relajación $D_{HS}(t)$ por Krylov con pasos grandes $\tau = 0,25$ ($N = 7$).